

西南交通大學
Southwest Jiaotong University

《软件系统综合课程设计》测试报告

设计题目: 软件系统综合课程设计练习 2 单元测试报告

学　　号:

姓　　名: 程泽凯

班　　级: 软件工程专业 2023 级

指导教师: 课程指导组

2026 年 6 月 17 日

摘要

本报告面向“西南交通大学校园论坛系统”完成软件系统综合课程设计练习 2。报告围绕本人负责的 3 个代码单元开展白盒单元测试设计，分别使用基本路径法、条件-判定覆盖和条件组合覆盖规划测试用例，并给出 JUnit 5 自动化脚本片段、测试环境、结果分析和缺陷记录。同时，报告补充火车票退票金额计算业务的黑盒场景法测试用例。

关键词：单元测试；白盒测试；基本路径法；条件覆盖；JUnit；退票场景法

目录

一、 实验基本信息 1

二、 实验目的与前提 1

三、 个人测试点总表 1

四、 白盒测试设计与分析 1

4.1 后台封禁用户逻辑 2

4.2 帖子举报创建逻辑 3

4.3 通知已读标记逻辑 6

五、 测试环境与执行方式 8

六、 测试结果与缺陷记录 8

七、 黑盒测试补充：火车票退票场景法 8

7.1 临近发车与办理渠道切换 8

八、 实验收获 9

一、 实验基本信息

项目	内容
实验名称	单元测试
项目名称	西南交通大学校园论坛系统
实验地点	软件工程实验室 X7507/7508
实验日期	2026 年 6 月 17 日
个人说明	本人负责后台治理和通知权限相关单元，重点检查管理员操作是否留痕、举报入口是否防滥用、通知是否不会越权读取。

二、 实验目的与前提

本次实验继续练习测试设计的基本概念，重点掌握白盒测试中的逻辑驱动法和路径方法。实验前已理解单元测试的基本思想，并在 Java 后端项目中准备 JUnit 5、Spring Boot Test、Mockito、Maven Surefire 等测试工具；覆盖率可通过 JaCoCo 扩展，静态分析可通过 SpotBugs 或 IDE 检查补充。

三、 个人测试点总表

序号	代码单元	方法位置	采用方法
1	后台封禁用户逻辑	AdminOpsService.banUser(Long, String)	基本路径法、条件-判定覆盖、条件组合覆盖
2	帖子举报创建逻辑	ReportService.createPostReport(Long, CreateReportRequest, String)	基本路径法、条件-判定覆盖、条件组合覆盖
3	通知已读标记逻辑	NotificationService.markRead(Long, String)	基本路径法、条件-判定覆盖、条件组合覆盖

四、 白盒测试设计与分析

本节对 3 个代码单元逐一分析。每个单元先摘录核心逻辑，再用文本控制流图标出判定节点；圈复杂度按“判定节点数 + 1”估算，随后列出基本路径、条件-判定覆盖和条件组合覆盖用例。

4.1 后台封禁用户逻辑

被测代码单元：`AdminOpsService.banUser(Long, String)`。该单元包含保留最后一个可用管理员的保护条件，适合做条件组合覆盖。

核心代码段摘录：

```
public User banUser(Long userId, String operatorName) {
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new NotFoundException("user not found"));
    if ("ADMIN".equals(user.getRole()) && Boolean.TRUE.equals(user.getEnabled())
            && userRepository.countByRoleAndEnabledTrue("ADMIN") <= 1) {
        throw new ApiException(400, "keep at least one admin");
    }
    user.setEnabled(false);
    User saved = userRepository.save(user);
    User operator = userService.getUserByUsername(operatorName);
    log(operator, "BAN_USER", "USER", userId, "ban user");
    return saved;
}
```

控制流图：

```
N1 start -> N2 user exists?
N2 no -> N3 not found -> End
N2 yes -> N4 role is ADMIN?
N4 no -> N8 disable user -> N9 log -> End
N4 yes -> N5 enabled?
N5 no -> N8 disable user -> N9 log -> End
N5 yes -> N6 admin count <= 1?
N6 yes -> N7 throw keep admin -> End
N6 no -> N8 disable user -> N9 log -> End
```

用户存在和最后管理员保护条件中三个子条件，共规划 5 条路径。

路径	路径说明
P1	用户不存在。
P2	目标是唯一可用管理员，拒绝封禁。
P3	目标是普通用户，封禁成功。
P4	目标是管理员但仍有其他可用管理员，封禁成功。
P5	目标管理员已禁用，再次封禁保持禁用并记录操作。

基本路径法测试用例：

编号	测试数据与条件	预期结果
BAN-P1	userId 不存在	NotFoundException
BAN-P2	role=ADMIN,enabled=true,count=1	ApiException 400
BAN-P3	role=USER	enabled=false，写操作日志
BAN-P4	role=ADMIN,enabled=true,count=2	enabled=false

编号	测试数据与条件	预期结果
BAN-P5	role=ADMIN,enabled=false	不触发最后管理员保护

条件-判定覆盖：

编号	覆盖条件	预期判定结果
BAN-CD1	isAdmin=T	管理员条件取真
BAN-CD2	enabled=T	启用状态条件取真
BAN-CD3	lastAdmin=T	保护条件取真
BAN-CD4	组合整体为假	允许封禁

条件组合覆盖：

编号	条件 1	条件 2	条件 3/4	组合期望
BAN-C1	T	T	T	拒绝封禁
BAN-C2	T	T	F	允许封禁
BAN-C3	T	F	-	允许封禁
BAN-C4	F	-	-	允许封禁

JUnit 脚本片段：

```
@Test
void banUserKeepsLastEnabledAdmin() {
    admin.setRole("ADMIN");
    admin.setEnabled(true);
    when(userRepository.countByRoleAndEnabledTrue("ADMIN")).thenReturn(1L);
    assertThrows(ApiException.class, () -> adminOpsService.banUser(admin.getId(), "root"));
}

@Test
void banUserDisablesNormalUserAndLogsOperation() {
    normal.setRole("USER");
    User saved = adminOpsService.banUser(normal.getId(), "root");
    assertThat(saved.getEnabled()).isFalse();
    verify(operationLogRepository).save(any(OperationLog.class));
}
```

缺陷与回归关注：未发现缺陷。该保护条件很关键，后续修改用户角色逻辑时应优先回归。

4.2 帖子举报创建逻辑

被测代码单元：`ReportService.createPostReport(Long, CreateReportRequest, String)`。该单元防止非法原因、自举报和重复举报，属

于后台治理流程入口。

核心代码段摘录：

```
public Report createPostReport(Long postId, CreateReportRequest request, String username) {
    User reporter = userService.getUserByUsername(username);
    Post post = postRepository.findById(postId)
            .orElseThrow(() -> new NotFoundException("post not found"));
    String reason = request.getReason().trim();
    if (!ALLOWED_REASONS.contains(reason)) {
        throw new ApiException(400, "bad reason");
    }
    if (post.getAuthor() != null && post.getAuthor().getId().equals(reporter.getId())) {
        throw new ApiException(400, "cannot report yourself");
    }
    if (reportRepository.existsByReporterAndPostAndStatus(reporter, post, STATUS_PENDING)) {
        throw new ApiException(400, "duplicate report");
    }
    Report report = new Report();
    report.setReporter(reporter);
    report.setPost(post);
    report.setReason(reason);
    return reportRepository.save(report);
}
```

控制流图：

```
N1 start -> N2 reporter exists -> N3 post exists?
N3 no -> N4 not found -> End
N3 yes -> N5 reason allowed?
N5 no -> N6 bad reason -> End
N5 yes -> N7 self report?
N7 yes -> N8 reject self report -> End
N7 no -> N9 duplicate pending?
N9 yes -> N10 reject duplicate -> End
N9 no -> N11 save report -> End
```

帖子存在、原因合法、是否举报自己、是否重复四个判定，V(G)=5。

路径	路径说明
P1	帖子不存在。
P2	举报原因不在允许集合内。
P3	举报自己的帖子。
P4	同一用户对同一帖子已有待处理举报。
P5	合法举报保存成功。

基本路径法测试用例：

编号	测试数据与条件	预期结果
REPORT-P1	postId 不存在	NotFoundException
REPORT-P2	reason= 随便写	ApiException 400

编号	测试数据与条件	预期结果
REPORT-P3	reporter=post.author	ApiException 400
REPORT-P4	exists pending=true	ApiException 400
REPORT-P5	reason= 垃圾广告，无重复	保存 Report

条件-判定覆盖：

编号	覆盖条件	预期判定结果
REPORT-CD1	postExists=F	帖子不存在
REPORT-CD2	reasonAllowed=F	原因不合法
REPORT-CD3	selfReport=T	拒绝自举报
REPORT-CD4	duplicate=T	拒绝重复举报
REPORT-CD5	全部满足	创建成功

条件组合覆盖：

编号	条件 1	条件 2	条件 3/4	组合期望
REPORT-C1	F	-	-	-；帖子不存在
REPORT-C2	T	F	-	-；非法原因
REPORT-C3	T	T	T	-；自举报
REPORT-C4	T	T	F	T；重复举报
REPORT-C5	T	T	F	F；成功

JUnit 脚本片段：

```
@Test
void createReportRejectsSelfReport() {
   post.setAuthor(reporter);
   request.setReason("垃圾广告");
   assertThrows(ApiException.class,
         () -> reportService.createPostReport(1L, request, "alice"));
}

@Test
void createReportSavesValidReport() {
   request.setReason("虚假信息");
```

```
    Report saved = reportService.createPostReport(1L, request, "bob");
    assertThat(saved.getReason()).isEqualTo("虚假信息");
    verify(reportRepository).save(any(Report.class));
}
```

缺陷与回归关注：未发现缺陷。建议前端原因下拉框和后端允许集合保持同步，减少非法原因提交。

4.3 通知已读标记逻辑

被测代码单元：`NotificationService.markRead(Long, String)`。该单元用 NotFoundException 隐藏他人通知存在性，适合验证权限分支。

核心代码段摘录：

```
public Notification markRead(Long id, String username) {
    User recipient = requireUser(username);
    Notification notification = notificationRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("notification not found"));
    if (notification.getRecipient() == null
            || !notification.getRecipient().getId().equals(recipient.getId())) {
        throw new NotFoundException("notification not found");
    }
    notification.setIsRead(true);
    return notificationRepository.save(notification);
}
```

控制流图：

```
N1 start -> N2 user exists?
N2 no -> N3 user not found -> End
N2 yes -> N4 notification exists?
N4 no -> N5 notification not found -> End
N4 yes -> N6 belongs to user?
N6 no -> N7 notification not found -> End
N6 yes -> N8 set read -> N9 save -> End
```

用户存在、通知存在、通知归属三个判定，V(G)=4。

路径	路径说明
P1	用户名不存在。
P2	通知不存在。
P3	通知存在但不属于当前用户。
P4	通知属于当前用户，标记已读成功。

基本路径法测试用例：

编号	测试数据与条件	预期结果
NOTICE-P1	username 不存在	NotFoundException

编号	测试数据与条件	预期结果
NOTICE-P2	notificationId 不存在	NotFoundException
NOTICE-P3	recipientId != currentUserId	NotFoundException
NOTICE-P4	recipientId == currentUserId	isRead=true 并保存

条件-判定覆盖：

编号	覆盖条件	预期判定结果
NOTICE-CD1	userExists=F	用户不存在
NOTICE-CD2	noticeExists=F	通知不存在
NOTICE-CD3	ownerMatch=F	不泄露他人通知
NOTICE-CD4	ownerMatch=T	标记已读

条件组合覆盖：

编号	条件 1	条件 2	条件 3/4	组合期望
NOTICE-C1	F	-	-	用户不存在
NOTICE-C2	T	F	-	通知不存在
NOTICE-C3	T	T	F	他人通知，仍返回不存在
NOTICE-C4	T	T	T	标记成功

JUnit 脚本片段：

```
@Test
void markReadDoesNotExposeOtherUsersNotification() {
    notification.setRecipient(otherUser);
    assertThrows(NotFoundException.class,
            () -> notificationService.markRead(9L, "alice"));
}

@Test
void markReadSetsReadFlag() {
    notification.setRecipient(alice);
    Notification saved = notificationService.markRead(9L, "alice");
    assertThat(saved.getIsRead()).isTrue();
    verify(notificationRepository).save(notification);
}
```

缺陷与回归关注：未发现缺陷。该分支体现了权限隐藏策略，需要和接口状态码契约保持一致。

五、 测试环境与执行方式

- 操作系统：Windows，项目目录为课程设计论坛后端。
- Java 环境：JDK 17，Spring Boot 3.2.0。
- 单元测试框架：JUnit 5，Spring Boot Starter Test，Mockito，AssertJ。
- 执行命令：进入 `jiaodaluntan/backend` 后运行 `mvn test`。
- 结果判定：异常分支检查异常类型和状态码，成功分支检查返回对象、保存动作、字段变化和关键仓储调用。

六、 测试结果与缺陷记录

本报告设计的白盒用例覆盖本人负责的 3 个单元。设计阶段发现的问题主要集中在交互提示、数据展示一致性和历史功能回归风险上；服务层核心逻辑通过路径拆分后边界比较清晰。实际提交时建议将报告中的脚本片段整理到对应的 `src/test/java` 测试类中，并与已有测试套件一起执行。

类型	位置	说明
已覆盖	3 个代码单元	每个单元均给出基本路径、条件-判定覆盖和条件组合覆盖用例。
测试脚本	JUnit 5 片段	报告中保留核心脚本，便于复制到项目测试目录继续执行。
缺陷记录	按单元列出	服务层未发现阻塞缺陷，个别页面展示和历史回归风险已标注。

七、 黑盒测试补充：火车票退票场景法

7.1 临近发车与办理渠道切换

我的黑盒测试假设票价为 214.50 元，主线放在临近发车时的渠道切换，同时检查大额票价下手续费进位规则。

场景法先确定基本流：张三提交退票申请，系统读取开车时间和当前退票时间，判断是否允许网上办理；随后按时间区间计算手续费，按 5 角规则取整，最后返回办理渠

道、手续费和退款金额。备选流包括已打印纸质票、晚于开车前 2 小时、退票时间落在 15 天/48 小时/24 小时边界，以及手续费尾数取整。

```
S0 start
S1 read ticket: departure=2026-06-30 07:52
S2 check paper ticket and hours before departure
S3 choose channel: online or station window
S4 choose fee rate: 0%, 5%, 10%, 20%
S5 round fee to nearest 0.5 yuan by railway rule
S6 output channel, fee, refund amount
```

编号	退票时间	纸质票	场景说明	办理渠道	预期结果
BB-CZK-01	2026-06-01 08:00	未打印	远大于 15 天	网上可退	手续费 0.00，退款 214.50
BB-CZK-02	2026-06-25 09:00	未打印	48 小时以上	网上可退	5% 为 10.725，取整 10.50，退款 204.00
BB-CZK-03	2026-06-28 23:00	未打印	24 到 48 小时之间	网上可退	10% 为 21.45，取整 21.50，退款 193.00
BB-CZK-04	2026-06-30 05:51	未打印	早于 2 小时 1 分钟	网上可退	20% 为 42.90，取整 43.00，退款 171.50
BB-CZK-05	2026-06-30 05:53	未打印	晚于 2 小时限制	窗口办理	20% 收 43.00，退款 171.50

这组用例没有和小组其他成员完全共用同一批时间点和票价，目的是让补充作业覆盖更多实际组合。真正落到系统测试时，票价可以参数化，时间点保持相对开车时间不变即可复用。

八、 实验收获

后台类代码最需要测试的是保护条件。封禁用户如果没有最后管理员保护，系统可能把自己锁死；举报如果不防重复和自举报，管理队列会很乱；通知如果把他人的通知返回成“无权访问”，反而可能泄露通知存在。练习路径法时，我觉得这些分支比成功路径更有价值，因为它们是系统保持可管理和可恢复的底线。